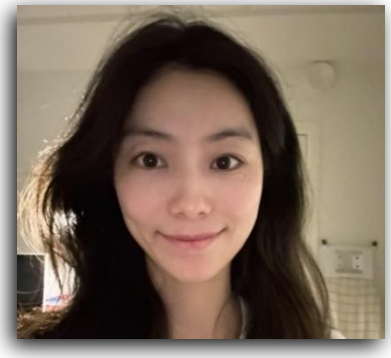
Opgave 5
ADC–PWM robotarm-styring
11/5/2026
Gruppe 1

Studienummer	Navn	Billede
S245011	Xingjia Yu	
S171678	Bilal Alali	

Contents

Introduktion 3
Specifikation. 3
Overordnet kravspecifikation og underliggende teknisk krav 3
Krav til ADC 3
Krav til PWM 4
C-Modul Diagram 6
Interrupt-diagram med forklaring 7
Forklaring 7
Flowchart int main() med forklaring 9
Forklaring 10
Test 10
Robotarm med video 11
Bilag 1 ClaudePrompts 11
Konklusion 12
Bilag 2: Koden 13
//main.c 13
//Timer.c 15
//intiADC.c 17

Introduktion

Formålet med denne opgave er at udvikle et struktureret embedded C-program til ATmega2560, som kan styre en robotarm med fire servomotorer ved hjælp af to joysticks Programmet benytter ADC-konvertering til at læse de analoge spændinger fra joystickene og PWM-signaler til at styre servoernes position.

Systemet er opbygget med flere moduler til henholdsvis ADC, PWM-timere, I2C og OLED-display. ADC'en bruges til at læse joystickenes fire akser via kanalerne A0–A3, mens Timer1, Timer3, Timer4 og Timer5 genererer PWM-signaler til de fire servoer. PWM-signalerne har en frekvens på ca. 50 Hz, som passer til standard servomotorer.

Specifikation.

Overordnet kravspecifikation og underliggende teknisk krav

Formålet med programmet er at styre en robotarm med fire servomotorer ved hjælp af to analoge joysticks. De fire joystick-akser forbindes til ADC-kanalerne A0, A1, A2 og A3 på ATmega2560. Hver ADC-kanal styrer én PWM-udgang, som er forbundet til en servomotor. Programmet opfylder dermed den udvidede kravspecifikation, hvor to joysticks styrer fire servoer via fire ADC-kanaler og fire PWM-signaler. Dette svarer til opgavens krav om styring af fire servoer via OC1A, OC3A, OC4A og OC5A

Krav til ADC

ADC'en bruges til at læse de fire analoge joystick-indgange A0–A3. ¨

Først vælges referencespændingen til ADC'en: ADMUX |= (1 << REFS0);

ADC'en aktiveres derefter i registeret ADCSRA: ADCSRA |= (1 << ADEN);

ADC-clockens prescaler sættes til 128 med disse bits: ADCSRA |= (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);

Da CPU-frekvensen er 16 MHz, bliver ADC-clockfrekvensen: 16 MHz / 128 = 125 kHz

Den ligger inden for det anbefalede interval.: 50 kHz < 125 kHz < 200 kHz

ADC-interrupt aktiveres med: ADCSRA |= (1 << ADIE); der laves et interrupt, når en ADC-konvertering er færdig. Når ADC-konverteringen er færdig, kaldes ADC-interruptet:

ISR(ADC_vect)

Den første ADC-konvertering startes med: ADCSRA |= (1 << ADSC);

Systemet læser ADC-kanalerne en ad gangen ved hjælp af Timer0 interrupt. Timer0-clock bliver efter prescaling 16 MHz / 64 = 250 kHz. En Timer0-tælling tager: 1 / 250 kHz = 4 µs. Da

OCR0A = 249, tæller timeren fra 0 til 249, altså 250 tællinger. Derfor kommer Timer0 compare-match interrupt hvert: 250 × 4 µs = 1000 µs = 1 ms. Og fordi koden skifter mellem fire kanaler: A0 -> A1 -> A2 -> A3 -> A0 ... bliver hver kanal samplet hvert: 4 × 1 ms = 4 ms

Krav til PWM

Timeren sættes op til at virke i phase and frequency correct PWM mode ved at sætte bitten WGM33 til 1 i registeret TCCRnB.

Timer1, Timer3, Timer4 og Timer5 bruges til at generere PWM-signaler til de fire servomotorer.

```
initTimer1(20000);

initTimer3(20000);

initTimer4(20000);

initTimer5(20000);
```

CPU-clock på 16 MHz og en prescaler på 8. Timerens frekvens bliver derfor: 16 MHz / 8 = 2 MHz. Tiden for et timer-tick bliver: 1 / 2 MHz = 0,5 µs, så hver timer-tick tager 0,5 µs. Men fordi signalet i phase and frequency correct PWM påvirkes både ved op- og nedtælling Derfor bliver den samlede PWM-periode:

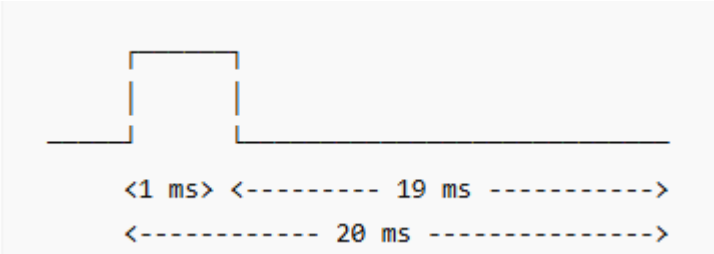
2 × ICRn × 0,5 µs = 2 × 20000 × 0,5 µs = 20000 µs = 20 ms

Værdien 20000 bruges som timerens TOP-værdi. Timerne er sat op til at lave PWM med en periode på ca. 20000 µs, hvilket svarer til 20 ms. Det giver en PWM-frekvens på: 1 / 20 ms = 50 Hz. Dette passer til almindelige servomotorer,

Timerne sættes op i fasekorrekt eller fase- og frekvenskorrekt PWM-mode. I denne type PWM tæller timeren op til TOP-værdien og derefter ned igen. PWM-udgangen styres af output compare-registeret, for eksempel OCR1A for Timer1.

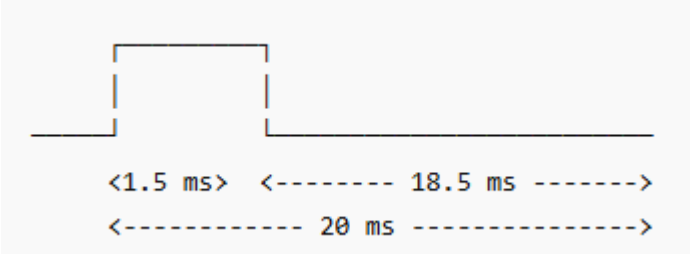
Minimumposition, OCRxA = 1000

HIGH: 2 × 1000 × 0,5 µs= 1,0 ms. Duty cycle beregnes som:1ms/20ms=5%
LOW: 19,0 ms



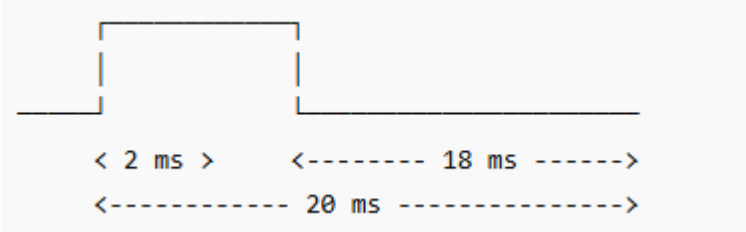
Neutralposition, OCRxA = 1500

HIGH: 2 × 1500 × 0,5 µs =1,5 ms. Duty cycle beregnes som:1.5ms/20ms=7.5%
LOW: 18,5 ms

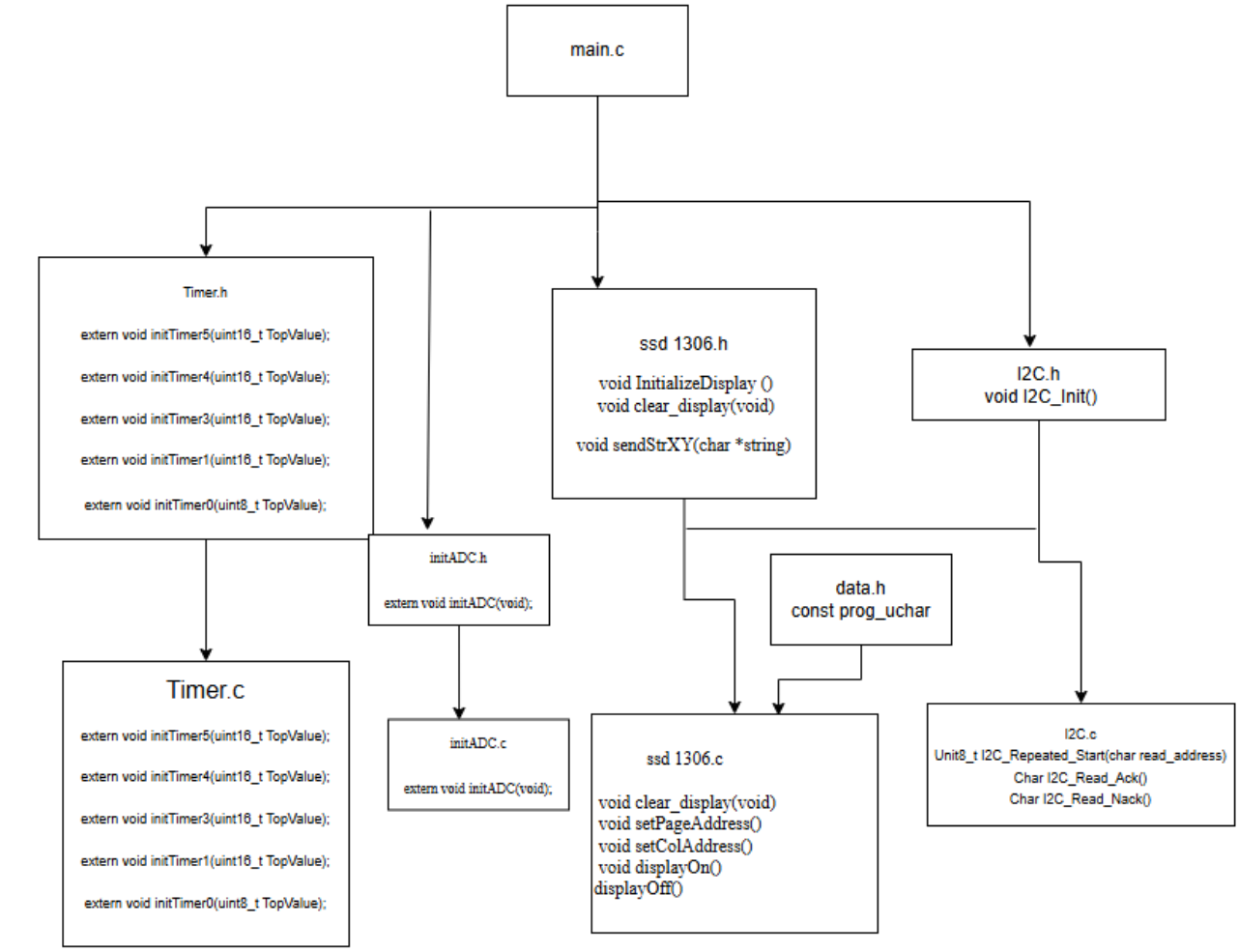


Maksimumposition, OCRxA = 2000

HIGH: 2 × 2000 × 0,5 µs =2,0 ms. Duty cycle beregnes som:1ms/20ms=10%
LOW: 18,0 ms

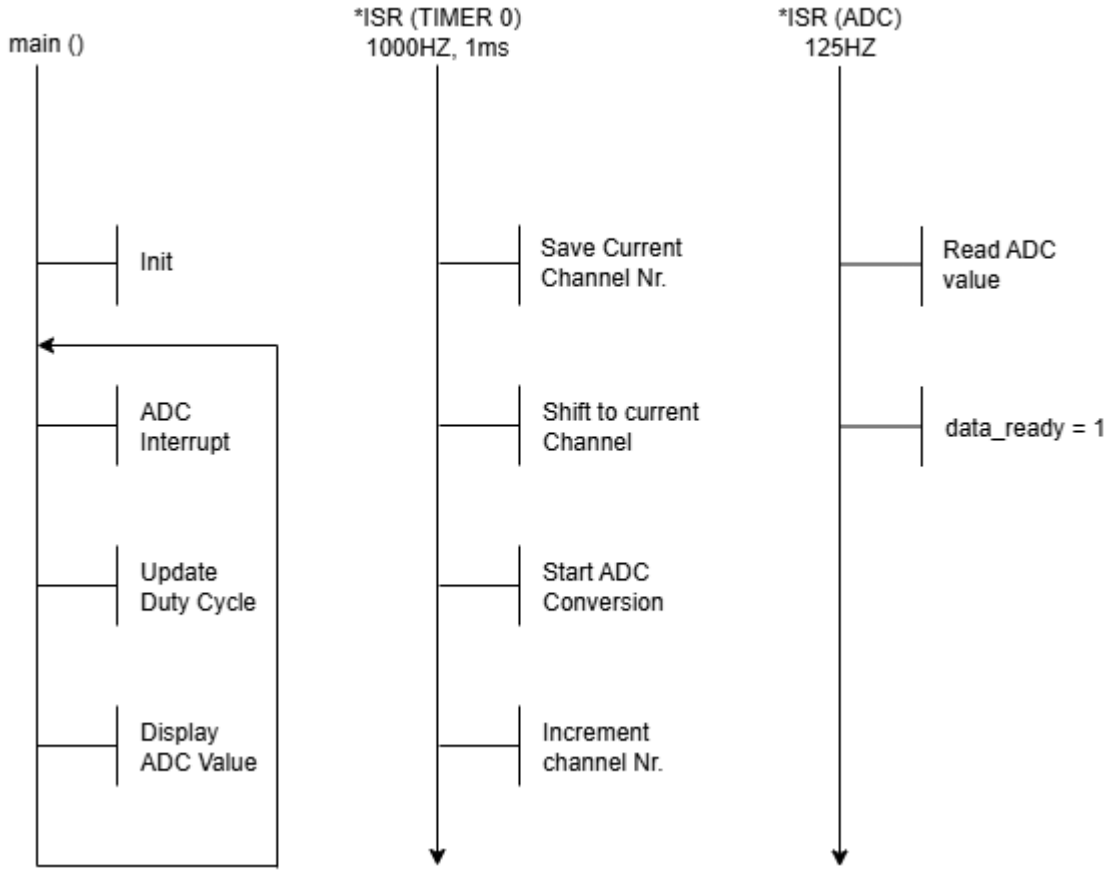


C-Modul Diagram



Interrupt-diagram med forklaring

I dette diagram kan man se paralleliteten mellem afbrydelserne (interrupts) og main-funktionen i C-main-modulet.



Forklaring

main() starter med at konfigurere alle komponenter – OLED, Timer 0, 1, 3, 4, 5 og ADC – og går derefter ind i en uendelig løkke. I løkken venter den på flaget data_ready fra ADC (sample rate er 1ms), opdaterer duty cycle for den tilsvarende kanal og viser de aktuelle ADC-værdier fra de fire joysticks hver 100 ms. Bemærk, at samplingsraten for hver kanal er 250 Hz (4 ms), fordi de fire kanaler skiftes hver 1 ms. Da visningen på OLED er langsom, nedsættes frekvensen for visning til 100ms adgangen, hvilket ikke påvirker de øvrige processer.

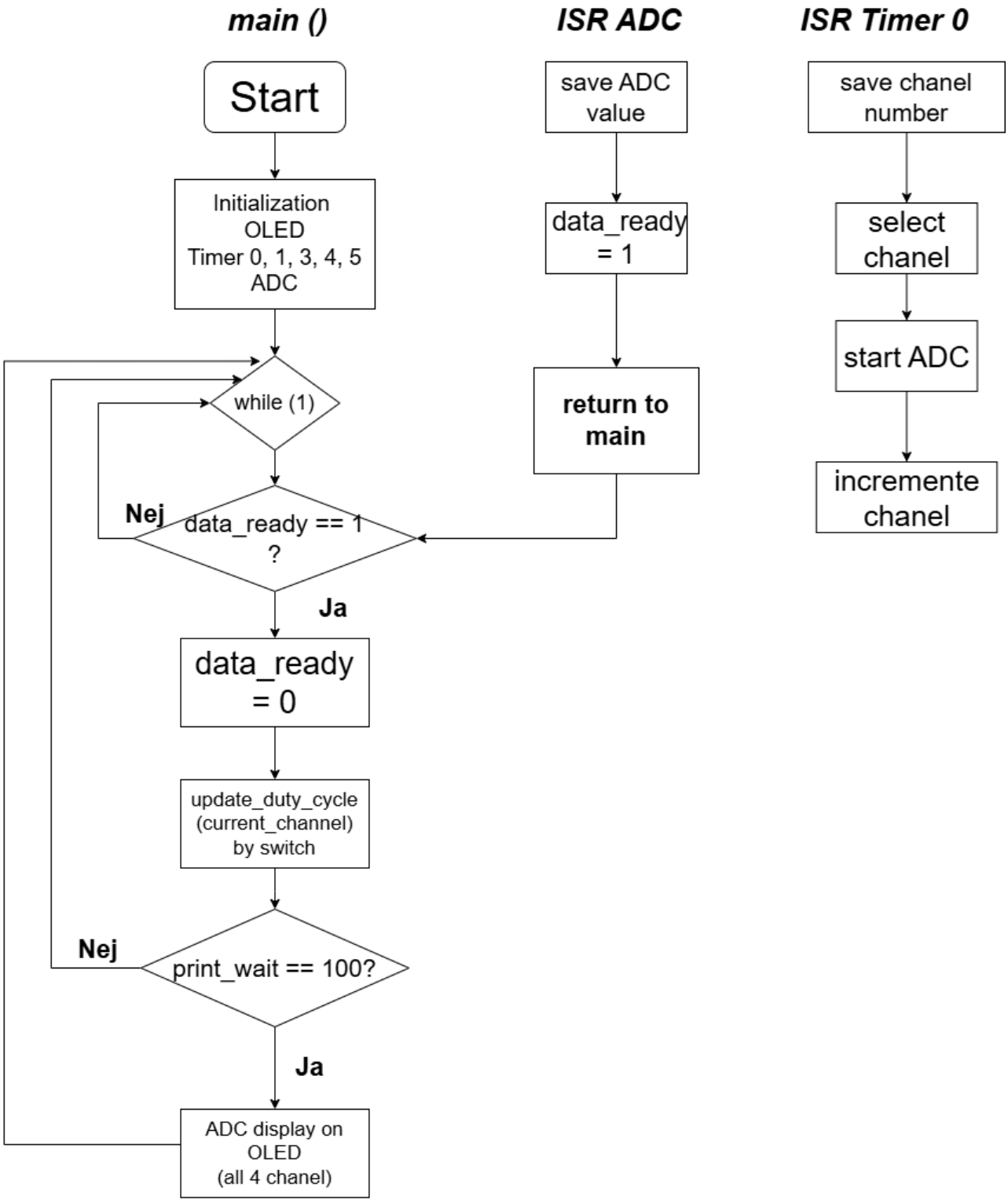
ISR (TIMER 0) — afbrydes hver 1 ms. Prescalling af Timer0 er 64. Og det svarer til frekvens 16MHZ/64 = 250000HZ. Det betyder at det tager 1/250000s = 0,004ms til at tælle 1 tick adgangen. Timer0 tæller fra 0 til 249, som svarer til 250*0,004ms = 1ms i total. ISR afbryder derfor hver 1ms. Under ISR af Timer0 startes en ADC-konvertering for den aktuelle kanal (0 til

3). Kanalnummeret gemmes, og derefter øges det med 1 til næste konvertering. Når kanalnummeret når 3, sættes det tilbage til 0 ved næste opdatering. Denne afbrydelse styrer sample rate for ADC'en.

ISR (ADC) — 125KHZ. ADC konfigureres til single conversion mode, som kræver ADSC sættes til 1 for at starte en konversion. Efter den første gang konversion tager det 13.5 clk signaler til at convertere analog signal til digital signal adgangen. Under ISR'en modtages ADC værdien. Og data_ready sættes til 1. I main-løkken opdages data_ready og tilsvarende duty cycle opdateres.

De to ISR'er kommunikerer med main() via volatile-flag for at sikre, at delte data altid læses direkte fra hukommelsen uden compiler-optimering.

Flowchart int main() med forklaring



Forklaring

Programmet består af en main-funktion og to interrupt service routines.

Step 1: Initialisering: OLED, Timer 0, 1, 3, 4, 5 and ADC.

Step 2: sei() — globale interrupts aktiveres.

Step 3: while(1)-løkken starter med at tjekke om flaget data_ready er sættes. Ekstern interrupt af ADC arbejder med at sætte flaget til 1 efter ADC-konversion er færdig. I main nulstilles data_ready, opdateres tilsvarende duty cycle. En switch function benyttes til at vælge den tilsvarende PWM.

Step 4: print_wait incrementeres en adgangen fra 0 til 100, når flaget sættes. Når print_wait bliver til 100, printes alle fire ADC værdierne fra to joysticks på OLED. Visningen på OLED tager lange tid, hvilket kan påvirker de øvrige dele og dermed skal kontrolleres ved print_wait.

Timer0 kontrollerer sample rate. Single conversion mode kræver at ADSC sættes til 1 hver gang. Idet ISR af Timer0 sker hver 1ms pga. 1000HZ, så er sample rate af ADC er 1ms. Men idet ADC kun kan læse værdien fra en kanal adgangen og der er fire kanaler i total, så er sample rate til hver kanal er 1000HZ/4 = 250HZ.

Test

Del test 1: ADC-værdier vist på OLED

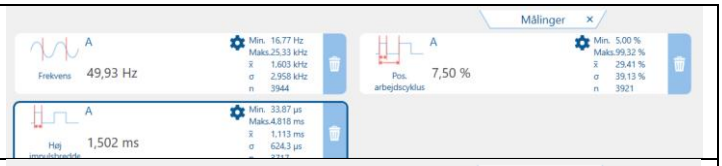
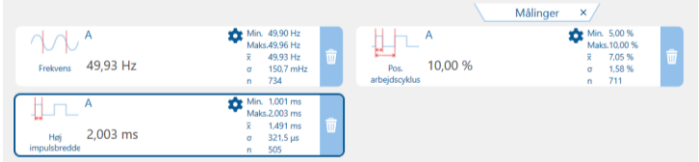
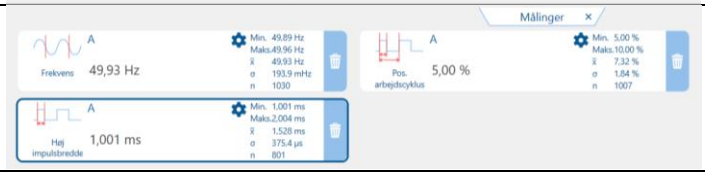
Joystick	Retning	Kommentar
Left	X-aksen	ADC-værdien på x-aksen går fra 0 til 1023, når det bevæges fra yderst til venstre til yderst til højre. Når joysticket står i midterposition, svinger værdien mellem 503 til 511.
	Y-aksen	ADC-værdien på y-aksen går fra 0 til 1023, når det bevæges fra yderst til venstre til yderst til højre. Når joysticket står i midterposition, svinger værdien mellem 500 og 506.
Right	X-aksen	ADC-værdien på x-aksen går fra 0 til 1023, når det bevæges fra yderst til venstre til yderst til højre. Når joysticket står i midterposition, svinger værdien mellem 513 og 519.
	Y-aksen	ADC-værdien på y-aksen går fra 0 til 1023, når det bevæges fra yderst til venstre til yderst til højre. Når joysticket står i midterposition, svinger værdien mellem 495 og 519.

Konklusion ad del test 1: Aflæsningen af joystickets midterværdi har en vis usikkerhed, især for x-aksen på det venstre joystick. Dette bør tages i betragtning ved fremtidig design af PWM, hvad angår dead zone-området for duty cycle

Del test 2: PWM med pulsbredde vist på Picoscop

PORT 11 (OCR1A) - left joysticks x-aksen

Position af joystick	Duty Cycle	Frekvens af PWM	On stage	Måling på Picoscop

I midten	7.5%	49,93HZ	1,502ms	
Drejer til højre	7.5% → 10%	49,93HZ	2,003ms	
Drejer til venstre	7.5% → 0%	49,93HZ	1,001ms	

Konklusion ad del test 2:

Forventet $f_{PWM} = \frac{f_{cpu}}{N*2*TOP} = \frac{16M}{8*2*20000} = 50HZ$, afvigelsen: $\frac{49{,}93-50}{50} = -0{,}14\%$

PW-range: 1,00ms til 2,003ms

Del test 3: samlet test af servo

PORT 11 (OCR1A) - left joysticks x-aksen

Efter tilslutning af servoen til pin 11 stod servobladet i midterpositionen (90 grader). Når joysticket blev skubbet mod højre, bevægede servoen sig jævnt til 30 grader og stoppede. Når joysticket blev skubbet mod venstre, bevægede servoen sig jævnt til 150 grader og stoppede.

Robotarm med video

Tjekke venligst uploadede videoen.

https://youtu.be/CmyP_95AZYM

Bilag 1 ClaudePrompts

Jeg har ikke brugt Claude Code til at skrive kode. Vi har i stedet brugt Claude til at forstå kode, kodeeksempler og betydningen af ATmega-periferi-konfigurationer. Nedenfor er en samling af spørgsmål.

#	Spørgsmål
1	What is the difference between pulse width and PWM period? If a datasheet specifies a maximum pulse width of 20ms, does this mean the maximum period is 20ms or the maximum HIGH duration is 20ms?
2	In Phase and Frequency Correct PWM mode on ATmega2560 Timer1, why is the output PWM frequency exactly half of the prescaled timer frequency, and how does the up-down counting affect this?

3	Why does including OLED sendStrXY and UART putstrUART0 inside every data_ready flag check cause the servo control to become unresponsive, and how can a print counter solve this?
4	How should the ADC channel switching be implemented in the Timer0 ISR so that current_channel is saved before switching to the next channel, ensuring the ADC ISR stores data in the correct array index?
5	What is the effect of the servo dead band width of 5us on the minimum OCR step size required to guarantee servo movement, and how does prescaler selection affect the number of controllable positions?

Konklusion

I denne opgave er der udviklet et embedded C-program til ATmega2560, som kan styre fire servomotorer på en robotarm ved hjælp af to joysticks. Programmet læser fire analoge indgange fra ADC-kanalerne A0–A3 og bruger værdierne til at ændre pulsbredden på fire PWM-signaler.

Samlet set opfylder programmet hovedformålet med opgaven, da det kan læse joystickværdier, generere PWM-signaler og styre flere servoer på en robotarm. Arbejdet har givet forståelse for ADC, timers, interrupts, PWM og modulopbygget programmering i embedded C.

Bilag 2: Koden

//main.c

```
#define F_CPU 16000000UL //frequency of CPU

#include <avr/io.h>
#include <stdio.h>
#include <avr/interrupt.h>
#include <util/delay.h>

#include "timer.h"
//#include "UART0.h"
#include "initADC.h"
#include "I2C.h"  //include library for i2c driver
#include "ssd1306.h" //include display driver



void Channel_Select(uint8_t channel){

            ADMUX = (ADMUX & 0b11100000)|channel;//shift channel
}

volatile int current_channel = 0;
volatile int channel = 0;
ISR (TIMER0_COMPA_vect){

            current_channel = channel;

            Channel_Select(channel);

            ADCSRA |= (1<<ADSC);  //start conversion for next time data reading

            if(channel < 3){
                        channel++;
                        } else {
                        channel = 0;
            }

}

volatile int data[4] = {0,0,0,0};
volatile int data_ready = 0;
ISR (ADC_vect){

             data[current_channel] = ADC;
             data_ready = 1;

}

void update_duty_cycle (int current_channel, int sample){

            //hold the last position of each channel
            static int last3 = 1500;
```

```
            static int last2 = 1500;
            static int last1 = 1500;
            static int last0 = 1500;

            //hold the position of the current channel
            int last = 0;

            switch (current_channel){
                        case 0: last = last0;
                        break;
                        case 1: last = last1;
                        break;
                        case 2: last = last2;
                        break;
                        case 3: last = last3;
                        break;
            }

            if (sample > 525 && sample <= 1023) {
                        last+=5;
            } else if (sample < 490 && sample >= 0) {
                        last-=5;
            }

            if (last > 2000) last = 2000;
            if (last < 1000) last = 1000;

            switch (current_channel){
                        case 0: OCR1A = last;
                        last0 = last;
                        break;
                        case 1: OCR3A = last;
                        last1 = last;
                        break;
                        case 2: OCR4A = last;
                        last2 = last;
                        break;
                        case 3: OCR5A = last;
                        last3 = last;
                        break;
            }

}

int main(void)
{
            //OLED initiation
            _i2c_address = 0X78; // write address for i2c interface
            I2C_Init();  //initialize i2c interface to display
            InitializeDisplay(); //initialize  display
            clear_display();   //use this before writing you own text

            //initialize timer & ADC & UART
            initTimer1(20000);
            initTimer3(20000);
            initTimer4(20000);
            initTimer5(20000);
            initADC();
            //uart0_Init(MYUBRRF);
```

```
            initTimer0(249);


            sei(); //enable global interrupt

            //display ADC value on OLED
            char LX[10];
            char LY[10];
            char RX[10];
            char RY[10];

    while (1)
            {
            if (data_ready == 1){

            //set flag back to 0 and convert data to string
            data_ready = 0;
            switch (current_channel){
            case 0: update_duty_cycle (0, data[0]);

            break;
            case 1: update_duty_cycle (1, data[1]);

            break;
            case 2: update_duty_cycle (2, data[2]);

            break;
            case 3: update_duty_cycle (3, data[3]);

            break;
            }

            static int print_wait = 0;
            print_wait++;
            if (print_wait >= 100){
            print_wait = 0;
            sprintf(LX, "LX: %04d", data[0]);
            sendStrXY(LX, 0, 0);
            sprintf(LY, "LY: %04d", data[1]);
            sendStrXY(LY, 1, 0);
            sprintf(RX, "RX: %04d", data[2]);
            sendStrXY(RX, 2, 0);
            sprintf(RY, "RY: %04d", data[3]);
            sendStrXY(RY, 3, 0);
            }

            }
    }
}
```

//Timer.c

```
#include <avr/io.h>
```

```
#include <avr/interrupt.h>

void initTimer0(uint8_t TopValue){
            //mode CTC with OCRn as Top Value + prescaler 64
            TCCR0A |= (1<<WGM01);
            TCCR0B |=(1<<CS00)|(1<<CS01);

            //OCR1A holds TopValue
            OCR0A = TopValue;

            //enable OCI
            TIMSK0 |= (1<<OCIE0A);
}
void initTimer1(uint16_t TopValue){
            //mode PWM phase and frequency correct with ICR as Top Value +
prescaler 8 + OCR1A 1500
            TCCR1B |= (1<<CS11)|(1<<WGM13);

            //clear OC1A on compare match
            TCCR1A |= (1<<COM1A1);

            //count up from 0 to 20000 and back to 0
            TCNT1 = 0;
            ICR1 = TopValue;

            //compare value for duty cycle 1st gang
            OCR1A = 1500;

            //set PWM output
            DDRB |= (1<<PB5);
}

void initTimer3(uint16_t TopValue){
            //mode PWM phase and frequency correct with ICR as Top Value +
prescaler 8 + OCR3A 1500
            TCCR3B |= (1<<CS31)|(1<<WGM33);

            //clear OC3A on compare match
            TCCR3A |= (1<<COM3A1);

            //count up from 0 to 20000 and back to 0
            TCNT3 = 0;
            ICR3 = TopValue;

            //compare value for duty cycle 1st gang
            OCR3A = 1500;

            //set PWM output
            DDRE |= (1<<PE3);
}

void initTimer4(uint16_t TopValue){

    //mode PWM phase and frequency correct with ICR as Top Value + prescaler 8 +
OCR3A 1500
    TCCR4B |= (1<<CS41)|(1<<WGM43);

    //clear OC4A on compare match
    TCCR4A |= (1<<COM4A1);
```

```
    //count up from 0 to 20000 and back to 0
    TCNT4 = 0;
    ICR4 = TopValue;

    //compare value for duty cycle 1st gang
    OCR4A = 1500;

    //set PWM output
    DDRH |= (1<<PH3);
}
void initTimer5(int TopValue){

    //mode PWM phase and frequency correct with ICR as Top Value + prescaler 8 +
OCR3A 1500
    TCCR5B |= (1<<CS51)|(1<<WGM53);

    //clear OC5A on compare match
    TCCR5A |= (1<<COM5A1);

    //count up from 0 to 20000 and back to 0
    TCNT5 = 0;
    ICR5 = TopValue;

    //compare value for duty cycle 1st gang
    OCR5A = 1500;

    //set PWM output
    DDRL |= (1<<PL3);
}
/*example for delay 1s by using corresponding ISR
volatile int Round = 0;
volatile char DelayFlag = 0;
ISR(TIMER1_COMPA_vect){
    Round++;
    if (Round == 1000){
        DelayFlag = 1;
        Round = 0;
    }
}*/
```

//intiADC.c

```
#include <avr/io.h>

void initADC(){

           ADMUX |= (1<<REFS0);//5V ref voltage

           ADCSRA |= (1<<ADEN)|(1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0) ;//enable ADC
with 128 prescaler => clk = 125 kHz

           ADCSRA |= (1<<ADIE); //enable interrupt

           ADCSRA |= (1<<ADSC);//start first time sample

}
```